



SKILL-USE: Can LLMs Actually Use Skills in Agentic Harnesses?

Jinyi Han^{1,*} Yuanjian Xu^{2,*} Ying Liao³ Xinyi Wang³ Zishang Jiang³
Zixiang Di¹ Fanyang Lu⁴ Zhichao Hu^{4,†} Yanghua Xiao^{3,†}

¹East China Normal University ²Hong Kong University of Science and Technology

³Fudan University ⁴Tencent Hunyuan

*Equal contribution. †Corresponding authors.



<https://github.com/JinyiHan99/Skill-Use-Bench>

Abstract

Large language model (LLM) agents increasingly rely on skills, structured documents that specify when to act, which procedure to follow, and which tools are allowed. Existing evaluations mostly judge the quality of a skill or its contribution to task success, leaving unexamined whether an agent can recognize a relevant skill and apply it on its own. We introduce SKILL-USE, a benchmark that evaluates skill use under progressive disclosure, where an agent sees only a skill’s name and short description and must retrieve the full procedure before following it. SKILL-USE separates three facets of skill use. Trigger measures whether the agent invokes the relevant skill, Compliance measures how faithfully it follows the prescribed procedure, and Boundary measures whether it avoids forbidden operations. A Skill-Use (SU) score combines the three and credits execution only after the skill is triggered. SKILL-USE pairs 79 real skills with 177 executable tasks across nine domains, each grounded in real files, run in an isolated Docker sandbox, and scored by a trajectory-based rubric. Evaluating eight LLMs under two agent harnesses, we find that reliable skill use remains out of reach, as the strongest configuration reaches an SU of only 0.613. Triggering and procedural compliance fail as independent bottlenecks, and both scores and model rankings shift with the harness, so skill use behaves as a capability conditioned on the harness rather than a fixed property of the model.

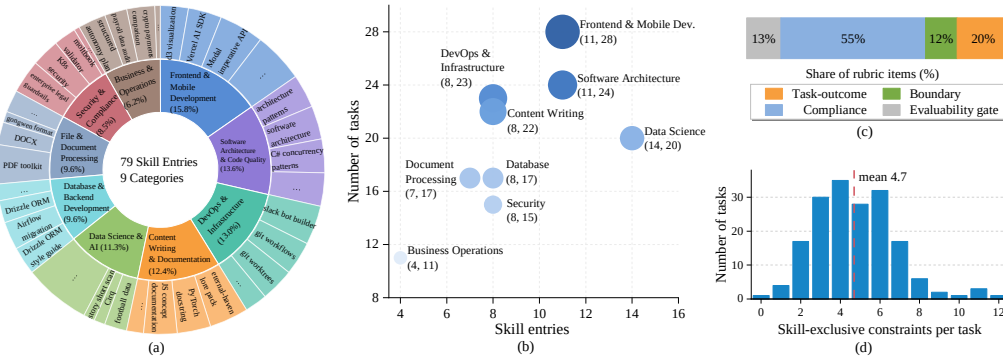


Figure 1: Composition and coverage of SKILL-USE. (a) Domain distribution of the 79 skills across nine categories, where the inner ring gives each category’s share and the outer ring lists representative skills. (b) Skill and task counts per domain, where each bubble marks one domain and reports its skill and task totals. (c) Composition of the rubric items by type. (d) Distribution of skill-exclusive requirements per task, our proxy for difficulty, where the dashed line marks the mean.

1 Introduction

Large language model (LLM) agents increasingly carry out complex, multi-step tasks in real software and web environments [1, 2]. Within the past year, skills have become an emerging interface for agent harnesses, popularized by Claude Code [3] and now appearing across systems such as Cursor, OpenClaw, and OpenAI Codex. Community repositories now host a fast-growing library of skills for diverse tasks [4, 5, 6, 7]. These systems and repositories treat a skill as a structured document that encodes reusable domain knowledge for accomplishing a class of tasks, specifying the situations in which it applies, the procedure to follow, and the tools and restrictions that govern its execution. The implicit assumption is that pairing a capable agent with the right skill is all it takes to solve the corresponding task.

The growing skill ecosystem translates into real capability only if agents can use the skills they are given. In the progressive-disclosure mechanism [3], the agent sees only a skill’s name and a short description; the full procedure is loaded only when the agent asks for it. To solve a task, the agent must first judge that a skill applies, retrieve it, and then follow it faithfully.

Existing benchmarks, however, do not evaluate all of these requirements together. Two lines of work come closest, but each covers only part of the process. Skill-centric benchmarks treat skills as artifacts and judge their intrinsic quality or their marginal contribution to task success [5, 6, 8, 9, 10]; their evidence is the final outcome rather than the consuming agent’s behavior. The closest effort scores adherence to the preconditions and orderings prescribed inside a skill, yet preloads the full skill and thereby bypasses retrieval altogether [11]. Instruction-following benchmarks score constraint compliance, but the constraints they target concern surface properties of the output such as format, length, or wording [12, 13, 14, 15, 16], and reach the agent verbatim in the prompt. A skill, in contrast, prescribes a domain procedure together with forbidden operations, and reaches the agent only through progressive disclosure. Neither line, therefore, addresses skill use as a process the agent must carry out on its own.

To fill this gap, we introduce **SKILL-USE**, a benchmark that evaluates skill use along three facets. Trigger measures whether an agent retrieves the relevant skill from its name and short description alone. Compliance measures adherence to the prescribed procedure, while Boundary measures avoidance of forbidden operations. By separating recognition, execution, and restraint, the benchmark reveals where skill use fails. Specifically, we construct SKILL-USE in three stages. We curate real skills and design executable tasks whose procedures leave observable traces and whose forbidden shortcuts are genuinely tempting. We then author trajectory-based rubrics for each task, and finally validate every skill-task-rubric triple end to end. Masked-skill screening removes generic requirements, and multi-agent adversarial review audits task scope and rubric verifiability.

SKILL-USE pairs 79 real skills from community repositories with 177 executable tasks across nine domains (Figure 1). Each task is grounded in real files and runs in an isolated Docker sandbox with full tool access; its complete trajectory is logged and scored using a multi-item rubric. We report the three facets separately and combine them into a gated Skill-Use (SU) score, where execution receives credit only after the relevant skill is triggered.

Evaluating eight state-of-the-art models under two agent harnesses, we find that reliable skill use remains out of reach. The strongest configuration reaches an SU of only 0.613. Triggering and procedural compliance break down as separate bottlenecks, since a model that recognizes a skill often still departs from its procedure while a model that could comply sometimes fails to trigger at all. Both absolute scores and model rankings shift once the harness changes, so skill use behaves as a capability conditioned on the surrounding harness rather than as a fixed property of the model [17, 18].

Contributions. In summary, we make the following contributions:

- We first formulate skill use as a distinct evaluation target and decompose it into three aspects, Trigger, Compliance, and Boundary, that capture recognition, procedural adherence, and restraint under progressive disclosure.
- We release SKILL-USE, a broad-coverage benchmark spanning nine domains, with 79 real-world skills and 177 sandboxed tasks graded by trajectory-based rubrics, together with a reusable three-stage construction pipeline.

- We conduct a systematic evaluation of eight frontier LLMs under two agent harnesses, yielding empirical insights into skill-use failure modes.

2 Related Work

Our work is most closely related to two lines of research: skill benchmarks and instruction-following benchmarks. Table 2 in Appendix A provides a detailed comparison.

Skill Benchmarks. Skills package reusable agent knowledge as progressive-disclosure documents [3, 5, 7]. SkillsBench [5] evaluates whether a provided skill improves task completion. SkillLearnBench [6] evaluates whether agents can generate reusable skills from experience through continual learning. Related work on skill libraries and continual skill acquisition focuses on organizing, producing, and refining such artifacts [8, 9, 10]. Collectively, these benchmarks focus on skill quality or final task success rather than whether an agent correctly uses a fixed skill. SLBench [11] is closest to our setting. It evaluates whether an agent respects logical relations within a skill, including preconditions, constraints, and overrides. However, preloading the full skill document leaves skill recognition and retrieval untested.

Instruction-Following Benchmarks. Instruction following measures whether LLMs can complete a task while satisfying user or system requirements. IFEval [12] formalizes this ability as compliance with verifiable constraints on length, format, keywords, and response structure. Later benchmarks broaden the scope to richer constraint types [13, 14, 15], system prompts [19], and multi-turn interactions [20]. AgentIF [16] further studies agentic instructions drawn from real system prompts. These benchmarks provide requirements directly in the prompt and therefore do not evaluate instruction following in the skill-use setting.

Agent Harness. Agent behavior depends not only on the base model but also on the surrounding harness that manages context, tools, and control flow. Recent studies show that the harness can account for more performance variance than the model and even reverse model rankings [17, 18]. Motivated by this, we evaluate skill use across multiple harnesses rather than treating it as a fixed property of the model.

In summary, SKILL-USE extends instruction-following evaluation to progressively disclosed skills. It also moves skill benchmarking beyond artifact quality and task success toward process-level evaluation in an open environment.

3 SKILL-USE Benchmark

We begin by formalizing the benchmark setting and evaluation protocol in Section 3.1. Section 3.2 then characterizes the benchmark’s composition and coverage. Finally, Section 3.3 details the three-stage construction pipeline.

3.1 Problem Formulation

Each evaluation instance asks whether an agent can recognize and correctly apply one relevant skill while solving a realistic task. An instance is a tuple $x = (s, q, \mathcal{E}, \mathcal{R})$ containing a skill s , a user task q , an executable sandbox $\mathcal{E}$, and a scoring rubric $\mathcal{R}$. The task consists of a user request and input assets such as a code repository, document, or dataset. The sandbox supplies the files and tools needed to execute the task. The rubric is hidden from the agent and is used only for evaluation.

Input. We represent the skill as $s = (m, p)$, where m contains its name, short description, and file path, and p contains its full procedure. The agent initially receives $(q, \mathcal{E}, m)$ as input. It does not receive p . Under the same progressive-disclosure interface [3], the agent must infer from the task and metadata that the skill is relevant, then retrieve the skill file to access its full procedure.

Output. Running the agent on this input produces an execution trajectory

$$\tau = (o_0, a_1, o_1, \dots, a_T, o_T) \quad (1)$$

and a terminal artifact state z_T . Here, o_0 is the initial context, a_k is the k -th action, and o_k is the observation returned after that action. Actions include reading files, invoking tools, and editing

artifacts. The state z_T contains the files produced or modified by the agent when execution ends. We denote the recorded output by $y = (\tau, z_T)$.

Scoring. Skill use unfolds in two stages: the agent must first recognize and retrieve the relevant skill, then apply its instructions faithfully. Trigger, formalized in Definition 1, captures the first stage. Compliance, defined in Definition 2, measures adherence to prescribed actions, while Boundary, defined in Definition 3, measures respect for prohibited actions. Together, the three dimensions characterize whether a skill is engaged and how faithfully it governs the subsequent execution.

Definition 1 (Trigger). For output $y = (\tau, z_T)$ and target skill s , Trigger is $g(y; s) := \mathbb{I}[\text{Retrieve}(s) \in \tau]$.

Definition 2 (Compliance). Let $\mathcal{R}^c$ be the set of prescribed skill requirements. Each item i has weight $w_i > 0$ and verifier $\phi_i(y) = 1$ iff the requirement is satisfied. Compliance is $C(y) := \frac{\sum_{i \in \mathcal{R}^c} w_i \phi_i(y)}{\sum_{i \in \mathcal{R}^c} w_i}$.

Definition 3 (Boundary). Let $\mathcal{R}^b$ be the set of skill prohibitions. Here $\phi_i(y) = 1$ iff the prohibited behavior is absent. Boundary is $B(y) := \frac{\sum_{i \in \mathcal{R}^b} w_i \phi_i(y)}{\sum_{i \in \mathcal{R}^b} w_i}$.

Finally, we consolidate the three dimensions into a single score that summarizes overall skill-use quality:

$$\text{SU}(y) = g(y; s) [\alpha C(y) + (1 - \alpha) B(y)], \quad (2)$$

We fix $\alpha = 0.7$ throughout the paper, giving Compliance the larger weight because it measures whether the agent enacts the skill’s prescribed procedure, while Boundary only guards against prohibited actions. Sensitivity to this choice is reported in Appendix E.6.

3.2 Benchmark Composition and Coverage

Domain Coverage. SKILL-USE contains 79 real skill entries paired with 177 executable tasks across nine domains, spanning software development, infrastructure, data science, databases, document and content processing, security, and business operations. As shown in Figure 1 (a) and (b), both skills and tasks are broadly distributed across these domains, with no single category dominating the benchmark.

Rubric Composition. Each task is graded by a hidden trajectory-level rubric. The 177 rubrics hold 1,314 scoring items, averaging 7.4 per rubric and ranging from 3 to 16. Figure 1(c) groups these items into four types. Evaluability gates carry zero weight and only flag whether a run is well-formed enough to score. Compliance and Boundary are skill-derived and together form the Skill-Use signal. Task-outcome items record whether the requested result was produced and remain outside Skill-Use.

Task Difficulty. We proxy difficulty by the number of skill-exclusive requirements per task, those that follow from the skill itself rather than from the task prompt or generic practice. As Figure 1(d) shows, each task carries 4.8 such requirements on average, with a tail reaching 12. Most tasks therefore expose several independent failure points rather than a single checkpoint.

3.3 Data Construction

SKILL-USE is constructed in three stages. We first collect and curate real skill documents and construct executable tasks for the retained skills. We then design a trajectory-based scoring rubric for each task. Finally, we validate every skill-task-rubric triple end to end. Figure 2 gives an overview of the pipeline.

Skill curation, task construction, and rubric construction each follow a written guideline that we refine iteratively. We start from a short draft, apply it, audit the artifacts, and fold each uncovered failure back as a new rule, repeating until audits surface no new failure. The multi-agent adversarial review enforces the task and rubric guidelines. Appendix B.6 reports the protocol and the failures behind each rule.

Skill and Task Construction. Candidate skills are collected from public GitHub repositories and two existing benchmarks, SkillsBench [5] and SkillLearnBench [4]. We use real documents to preserve the structures, workflows, and tool assumptions encountered in practice, and retain skills according to two criteria:

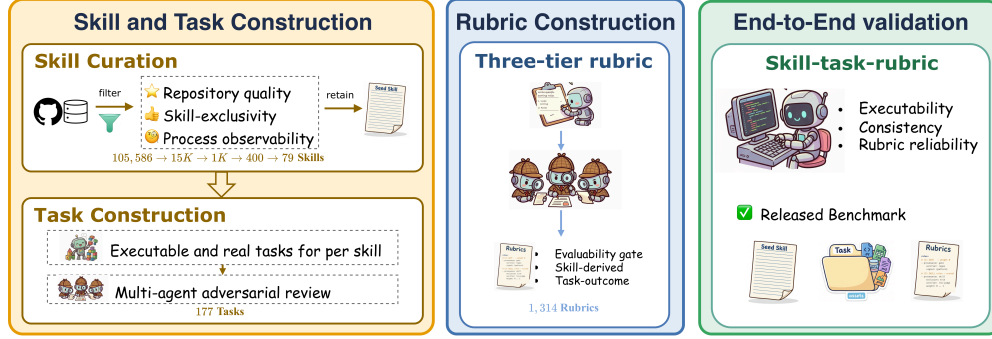


Figure 2: Data construction pipeline of SKILL-USE. The stages run left to right. Skill and Task Construction curates 79 skills from 105,586 candidates and builds one in-scope task per skill, giving 177 tasks. Rubric Construction attaches a three-tier rubric whose skill-derived items form the Skill-Use signal. End-to-End Validation checks each skill-task-rubric triple before release.

- **Skill exclusivity.** The skill must contain requirements that cannot be inferred from the task request or generic practice, allowing the scored behavior to be attributed to the skill itself.
- **Process observability.** The skill must constrain how the agent acts and leave verifiable evidence in its trajectory or artifacts, such as tool choices, action ordering, or prohibited operations.

We rank and sample candidates by repository quality and domain coverage, screen their process constraints, and apply a masked-skill check to remove generic requirements. Appendix B.1 reports the funnel and its thresholds. For each retained skill, we construct an executable task within its declared scope that exercises at least one skill-exclusive requirement, uses real assets, and hides the skill’s prescribed procedure from the prompt. Two filters guard task quality. The masked-skill check drops any requirement a strong model still meets from the prompt alone, so every scored requirement remains skill-exclusive. A cross-family adversarial review then requires reviewers from different model families to confirm each such requirement, and we apply its revisions or discard tasks that cannot be repaired. Detailed descriptions are provided in Appendix B.2 and Appendix B.4.

Rubric Construction. For each task, we construct a weighted rubric with three tiers. Evaluability gates determine whether the run can be scored, skill-derived items measure Compliance and Boundary, and task-outcome items record whether the requested result was produced. Only skill-derived items contribute to Skill-Use. We design these items according to two criteria:

- **Skill exclusivity.** Every scored item must trace to a distinct skill requirement that is not implied by the task prompt or generic practice.
- **Observable evidence.** Every item must be verified from a specific trajectory event or artifact. We prefer deterministic checks and use a scope-restricted LLM judge only when a requirement cannot be expressed reliably as a rule. Appendix B.3 specifies the rubric tiers and the judge.

Each rubric passes the same multi-agent adversarial review detailed in Appendix B.4, and we apply its revisions before end-to-end validation.

End-to-End Validation. Finally, we validate each assembled skill-task-rubric triple as a whole and retain only instances that satisfy three criteria:

- **Executability.** All required tools, dependencies, and assets are available in the isolated sandbox, allowing the task to run to completion under the evaluation harness.
- **Cross-component consistency.** Each task requirement traces to a specific skill requirement, and each rubric item targets an observable signal that the task can produce during execution.
- **Rubric reliability.** We replay each rubric on collected trajectories to identify inactive signals, unstable verifiers, format-sensitive failures, and leakage between Skill-Use and task-completion scoring.

4 Experiments

4.1 Experimental Setup

Models and harnesses. We evaluate eight LLMs from seven families, namely Claude Opus 4.7, Claude Opus 4.8, GPT-5.5, DeepSeek-V4-Pro, GLM-5.1, MiniMax-M3, Kimi-K2.6, and Qwen3.6-Max. Each model is evaluated under two agent harnesses, Claude Code (CC) and Codex. Every task runs in an isolated Docker sandbox. Sampling parameters and gateway routing are reported in Appendix D.

Metrics. We report Trigger, Compliance, Boundary, and their gated aggregate Skill-Use (SU), as defined in Section 3.1. Model-level metrics are means over evaluation trajectories. $\text{Comp.}^\dagger$ and $\text{Bound.}^\dagger$ restrict to traces that trigger the target skill. Rubric items judged by an LLM use GPT-5.4 at temperature 0, following the standard LLM-as-a-judge practice [21].

Harness	Model	Skill-Use				On Triggered Traces		
		Trigger	Compliance	Boundary	SU	Compliance [†]	Boundary [†]	SU ⁺
Claude Code	Claude Opus 4.7	0.966	0.609	0.712	0.599	0.606	0.709	0.637
	Claude Opus 4.8	0.940	0.625	0.704	0.597	0.638	0.710	0.660
	GPT-5.5 🏆	0.972	0.611	0.718	0.613	0.625	0.723	0.654
	DeepSeek-V4-Pro	0.324	0.439	0.658	0.250	0.588	0.699	0.621
	GLM-5.1	0.324	0.455	0.639	0.267	0.638	0.632	0.636
	MiniMax-M3	0.833	0.569	0.706	0.501	0.592	0.698	0.624
	Kimi-K2.6	0.337	0.447	0.580	0.190	0.576	0.464	0.542
	Qwen3.6-Max	0.446	0.475	0.625	0.318	0.590	0.650	0.608
Codex	Claude Opus 4.7	0.836	0.604	0.705	0.535	0.626	0.724	0.655
	Claude Opus 4.8 🏆	0.881	0.599	0.715	0.559	0.626	0.721	0.654
	GPT-5.5	0.966	0.486	0.700	0.503	0.490	0.709	0.556
	DeepSeek-V4-Pro	0.650	0.496	0.653	0.379	0.571	0.701	0.610
	GLM-5.1	0.706	0.525	0.617	0.421	0.599	0.616	0.604
	MiniMax-M3	0.864	0.475	0.632	0.446	0.497	0.655	0.544
	Kimi-K2.6	0.706	0.489	0.623	0.374	0.529	0.614	0.554
	Qwen3.6-Max	0.684	0.266	0.573	0.238	0.304	0.571	0.384

Table 1: **Main results on SKILL-USE across Claude Code and Codex.** All metrics are scaled to $[0, 1]$. Compliance and Boundary are averaged over all traces; their [†] variants are restricted to traces that trigger the target skill. 🏆 marks the highest-SU model under each harness.

4.2 Main Results

Table 1 reports SU together with its Trigger, Compliance, and Boundary components under both harnesses. We make two main observations:

🔍 Findings: Reliable skill use remains out of reach because triggering and conditional execution remain distinct bottlenecks.

The strongest configuration, GPT-5.5 under CC, achieves an SU of only 0.613. Missed triggers alone do not explain this gap. Even among triggered traces, the highest Compliance[†] reaches only 0.638. Boundary is higher than Compliance across all model-harness configurations, so models avoid forbidden actions more reliably than they complete the full prescribed workflow. Under CC, a group of open-weight middle-tier models trigger the target skill on only about a third of tasks, yet their conditional compliance is on par with models whose SU is twice as high. For them the missing capability is recognizing when the skill applies, not carrying it out. Under Codex, trigger rates concentrate in a narrower band and conditional compliance becomes the dominant source of between-model variation.

🔍 Findings: Skill-use performance and rankings depend on the harness because CC and Codex stress different sub-abilities.

Absolute scores and relative orderings both shift when the harness changes. The head of the ranking is occupied by GPT-5.5 under CC and by Claude Opus 4.8 under Codex; several models improve under Codex while others regress, and the correlation between the two per-model SU vectors is only moderate, as detailed in Appendix E.1. SU is therefore a property of a model-harness configuration, and rankings under one harness may not transfer to another. Moving from CC to Codex compresses

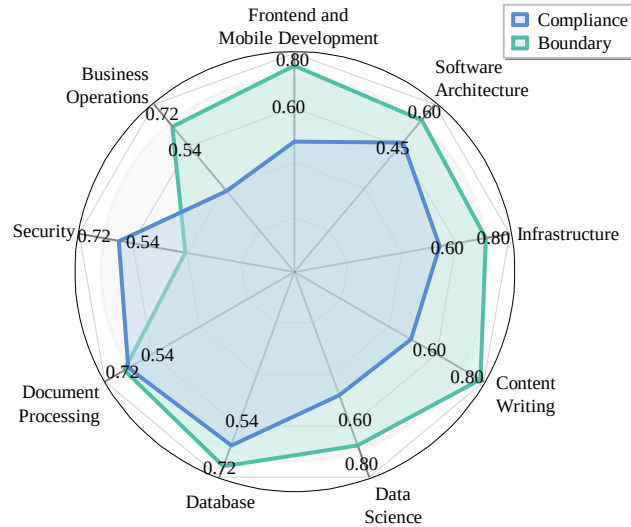


Figure 3: Compliance and Boundary scores by skill category (CC). Boundary exceeds Compliance in every category except Security & Compliance, where models follow the prescribed steps but fail to respect prohibitions.

the spread of Trigger while widening the spread of conditional Compliance. Codex lifts the weakest triggers toward the pack, yet some strong models lose ground on execution quality, with GPT-5.5’s Compliance[†] falling by more than a tenth. Codex thus lowers the barrier to selecting a skill but demands more sustained procedural execution once the skill is entered. Models whose strength is triggering benefit from CC, while models whose strength is execution retain more of their score under Codex.

4.3 How Skill Type Affects Skill-Use

To examine how skill-use performance varies across skill categories, Figure 3 reports category-level Compliance and Boundary.

Q Findings: Models follow concrete procedures more reliably than open-ended ones and generally avoid prohibitions more easily than they complete required workflows.

File processing, database, and data science skills have the highest Compliance. These skills typically prescribe a named tool, file format, or well-defined operation. Business analysis and software architecture score lower because their workflows involve open-ended planning. Boundary exceeds Compliance in almost every category, confirming that models suppress banned actions more easily than they execute every required step. Security and compliance is the sole exception. Models in this category achieve reasonable Compliance but the lowest Boundary, indicating that they carry out the safety steps yet still invoke the skill where restraint is needed.

4.4 How Skill Injection Mode Affects Skill-Use

To isolate the effect of skill injection, we pair two Codex runs on every (task, model) combination, changing only how the skill enters the agent’s context. In the native mode, the prompt lists the skill’s name, one-line description, and file path; the model must actively open the file to see the full procedure. In the preloaded mode, the entire skill document is inserted into the initial instructions so the procedure is visible from the start. Figures 4 and 5 report Trigger and SU under both modes.

Q Findings: Preloading primarily improves skill selection, revealing retrieval as the main bottleneck under native injection.

On all paired traces, preloading improves SU because it raises Trigger. The effect is largest for models whose native Trigger is weak and nearly disappears for models already near saturation. When

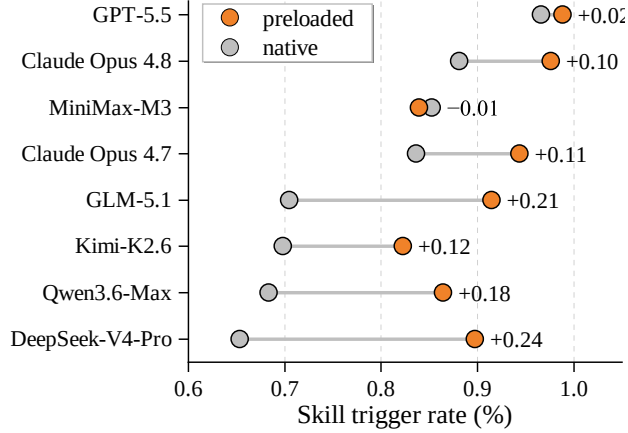


Figure 4: Per-model Trigger rate under both injection modes on paired traces. Preloading consistently raises Trigger.

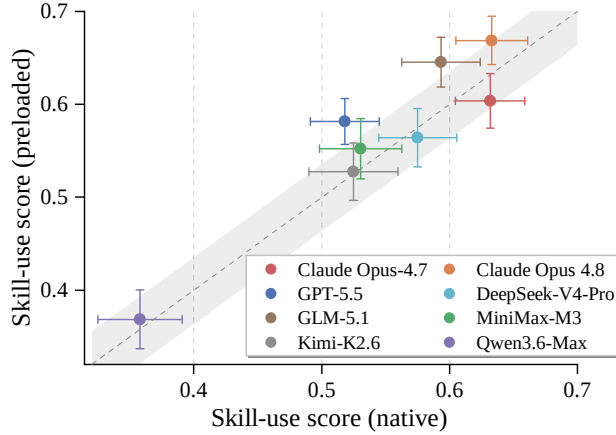


Figure 5: Native SU (x) against preloaded SU (y). Per-model SU restricted to traces that trigger under both modes. Clustering near the diagonal indicates that injection mode has little effect on execution quality after the skill is triggered.

both modes trigger, the SU gap becomes small and most model points stay close to the diagonal. Thus, showing the full skill text mainly helps the model recognize the skill as relevant; it does not materially improve rule following once the skill has been selected.

Native injection asks the model to act from a compact skill description before it sees the full procedure. The paired comparison shows that this extra decision, rather than downstream execution, accounts for most of the injection effect.

4.5 How Skill Library Size Affects Skill-Use

We test whether a larger skill library makes it harder to trigger the correct skill. Under CC, we install $N \in \{1, 10, 20, 30\}$ skills into the agent’s library. The target skill is always included; the remaining $N-1$ slots are filled with randomly sampled distractors. We label a run correct if the model selects the target skill, wrong if it selects a distractor, and none if it does not invoke any skill. The results are shown in Figure 6(a–b).

Findings: Adding distractors causes the main performance drop, driven more by missed triggers than by wrong-skill selections.

All models mostly select the target reliably in the single-skill setting. Moving to a ten-skill library causes the visible drop, especially for weaker models. After that point, the curves change little.

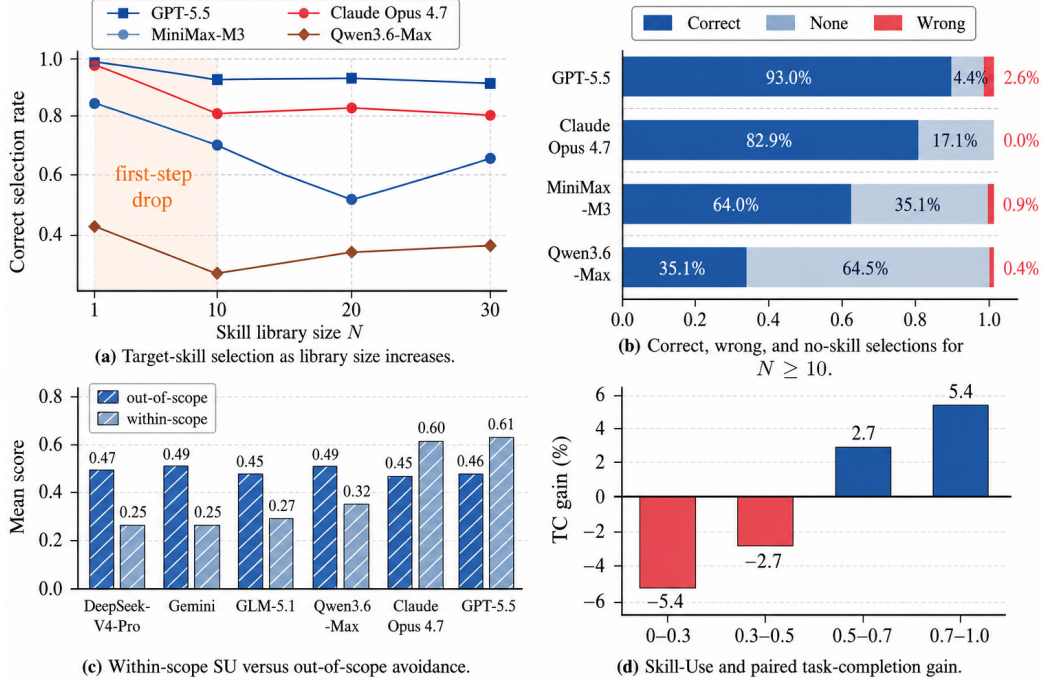


Figure 6: Skill selection, applicability, and task-completion effects under CC. (a–b) Increasing library size mainly raises no-skill outcomes rather than wrong-skill selections. (c) Within-scope Skill-Use and out-of-scope avoidance are not aligned across models. (d) Paired task-completion gain increases with Skill-Use; each triggered skill-enabled trace is paired with a no-library baseline on the same task and model.

Library growth creates an entry cost, not a steady decline. At larger library sizes, most failures are runs with no skill call. Wrong-skill calls are rare and occur mainly for broad, overlapping descriptions such as test-driven-development and software-architecture, as documented in Appendix E.3. Models often fail to decide that any skill is relevant enough to invoke.

4.6 How Models Avoid Unnecessary Skill Use

All previous experiments use tasks that fall within the target skill’s scope. Here we ask the question: can models refrain from invoking a skill when it does not apply? We construct 53 out-of-scope tasks whose topics overlap a skill but whose requested outputs do not require that skill’s procedure. We evaluate six models under CC and define an avoidance score as the fraction of out-of-scope tasks on which the model does not invoke the target skill. Figure 6(c) compares each model’s avoidance score with its within-scope SU.

Findings: Strong skill use does not guarantee restraint when skills do not apply.

The rank correlation between within-scope SU and out-of-scope avoidance is negative, so the models best at following a skill can be the worst at suppressing unnecessary invocations. The two abilities rest on separate judgments. One governs behavior after entering a skill, and the other governs whether to enter at all. The over-invocations are driven by topical overlap, mostly document-shaped requests whose topic matches the skill’s domain even though the output never needs its procedure, as detailed in Appendix E.4.

4.7 How Skill-Use Relates to Task Completion

We test whether following a skill’s procedure improves task completion, as shown in Figure 6(d). Under CC, we start from the 779 runs that trigger the target skill. For each run, we build a paired

baseline on the same task and model with the skill library disabled. We group the pairs by SU and report the mean paired gain $TC_{\text{skill}} - TC_{\text{base}}$ in each bin.

Q Findings: Whether a skill helps or hurts depends on both skill-use quality and skill type.

The paired gain turns from negative to positive near an SU score of 0.5. Below this point, the model uses the skill partially or incorrectly, and task completion drops below the no-skill baseline. Above it, the model follows the procedure closely, and the skill acts as a scaffold that raises completion. A partial procedure is worse than none because the model commits to a prescribed toolchain or format without carrying it through, and the resulting artifact is neither what the skill demands nor what the model would produce unaided. Adding a skill library is therefore not a free improvement; it pays off only when a model can execute the procedure and when the skill supplies an operational recipe rather than a stylistic prescription.

5 Conclusion

This paper introduces SKILL-USE, a benchmark for evaluating skill use in LLM agents under progressive disclosure. SKILL-USE decomposes skill use into Trigger, Compliance, and Boundary, and pairs 79 real skills with 177 executable tasks across nine domains. Using this benchmark, we evaluate eight frontier LLMs under two agent harnesses. The results show that skill use remains an open problem: no configuration achieves reliable performance on all three axes, and both scores and rankings depend on the harness rather than on the model alone. Our analysis further indicates that the main gap is retrieval under progressive disclosure, and that within-scope skill use does not transfer to restraint on out-of-scope tasks. We release SKILL-USE, together with its construction pipeline, to support future work on training and evaluating agents that use skills reliably.

References

- [1] Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, et al. Large language model agent: A survey on methodology, applications and challenges. *arXiv preprint arXiv:2503.21460*, 2025.
- [2] Frank Fangzheng Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, et al. Theagentcompany: benchmarking llm agents on consequential real world tasks. *Advances in Neural Information Processing Systems*, 38, 2026.
- [3] Yash Vishe, Rohan Surana, Xunyi Jiang, Zihan Huang, Xintong Li, Nikki Lijing Kuang, Tong Yu, Ryan A Rossi, Jingbo Shang, Julian McAuley, et al. Skill-r1: Agent skill evolution via reinforcement learning. *arXiv preprint arXiv:2605.09359*, 2026.
- [4] George Ling, Shanshan Zhong, and Richard Huang. Agent skills: A data-driven analysis of claude skills for extending large language model functionality. *arXiv preprint arXiv:2602.08004*, 2026.
- [5] Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, et al. Skillsbench: Benchmarking how well agent skills work across diverse tasks. *arXiv preprint arXiv:2602.12670*, 2026.
- [6] Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo FR Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong. Skilllearnbench: Benchmarking continual learning methods for agent skill generation on real-world tasks. *arXiv preprint arXiv:2604.20087*, 2026.
- [7] Hao Li, Chunjiang Mu, Jianhao Chen, Siyue Ren, Zhiyao Cui, Yiqun Zhang, Lei Bai, and Shuyue Hu. Organizing, orchestrating, and benchmarking agent skills at ecosystem scale. *arXiv preprint arXiv:2603.02176*, 2026.
- [8] Yuan Liang, Ruobin Zhong, Haoming Xu, Chen Jiang, Yi Zhong, Runnan Fang, Jia-Chen Gu, Shumin Deng, Yunzhi Yao, Mengru Wang, et al. Skillnet: Create, evaluate, and connect ai skills. *arXiv preprint arXiv:2603.04448*, 2026.
- [9] Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, et al. Autoskill: Experience-driven lifelong learning via skill self-evolution. *arXiv preprint arXiv:2603.01145*, 2026.
- [10] Huichi Zhou, Siyuan Guo, Anjie Liu, Zhongwei Yu, Ziqin Gong, Bowen Zhao, Zhixun Chen, Menglong Zhang, Yihang Chen, Jinsong Li, et al. Memento-skills: Let agents design agents. *arXiv preprint arXiv:2603.18743*, 2026.
- [11] Xuan Chen, Chengpeng Wang, Lu Yan, and Xiangyu Zhang. Slbench: Evaluating how llm agents follow logical relations in skills. *arXiv preprint arXiv:2607.09016*, 2026.
- [12] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.
- [13] Yuxin Jiang, Yufei Wang, Xingshan Zeng, Wanjuan Zhong, Liangyou Li, Fei Mi, Lifeng Shang, Xin Jiang, Qun Liu, and Wei Wang. Followbench: A multi-level fine-grained constraints following benchmark for large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4667–4688, 2024.
- [14] Yiwei Qin, Kaiqiang Song, Yebowen Hu, Wenlin Yao, Sangwoo Cho, Xiaoyang Wang, Xuansheng Wu, Fei Liu, Pengfei Liu, and Dong Yu. Infobench: Evaluating instruction following ability in large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 13025–13048, 2024.
- [15] Bosi Wen, Pei Ke, Xiaotao Gu, Lindong Wu, Hao Huang, Jinfeng Zhou, Wenchuang Li, Binxin Hu, Wendy Gao, Jiabin Xu, et al. Benchmarking complex instruction-following with multiple constraints composition. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, pages 137610–137645, 2024.
- [16] Yunjia Qi, Hao Peng, Xiaozhi Wang, Amy Xin, Youfeng Liu, Bin Xu, Lei Hou, and Juanzi Li. Agentif: Benchmarking large language models instruction following ability in agentic scenarios. *Advances in Neural Information Processing Systems*, 38, 2026.
- [17] Yunbei Zhang, Janet Wang, Yingqiang Ge, Weijie Xu, Jihun Hamm, and Chandan K Reddy. Stop comparing llm agents without disclosing the harness. *arXiv preprint arXiv:2605.23950*, 2026.
- [18] Yilun Yao, Xinyu Tan, Chao-Hsuan Liu, Yaoming Li, Zhengyang Wang, Wenhan Yu, Zhewen Tan, Yuxuan Tian, Guangxiang Zhao, Lin Sun, et al. Harness-bench: Measuring harness effects across models in realistic agent workflows. *arXiv preprint arXiv:2605.27922*, 2026.
- [19] Yanzhao Qin, Tao Zhang, Yanjun Shen, Wenjing Luo, Yan Zhang, Yujing Qiao, Zenan Zhou, Wentao Zhang, Bin CUI, et al. Sysbench: Can llms follow system message? In *The Thirteenth International Conference on Learning Representations*, 2024.

-
- [20] Yun He, Di Jin, Chaoqi Wang, Chloe Bi, Karishma Mandyam, Hejia Zhang, Chen Zhu, Ning Li, Tengyu Xu, Hongjiang Lv, et al. Multi-if: Benchmarking llms on multi-turn and multilingual instructions following. *arXiv preprint arXiv:2410.15553*, 2024.
 - [21] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623, 2023.
 - [22] Zekun Li, Shinda Huang, Jiangtian Wang, Nathan Zhang, Antonis Antoniadis, Wenyue Hua, Kaijie Zhu, Sirui Zeng, Chi Wang, William Yang Wang, et al. Sopbench: Evaluating language agents at following standard operating procedures and constraints. *arXiv preprint arXiv:2503.08669*, 2025.

Appendix Contents

A Benchmark Comparison	13
B Construction Pipeline	13
B.1 Skill Selection	13
B.2 Task Construction	15
B.3 Rubric and Judge	15
B.4 Adversarial Review	15
B.5 Release Validation	16
B.6 Guideline Development	16
C Human Quality Audit	17
D Experimental Setup	18
E Additional Analysis	18
E.1 Cross-Harness Behaviour	18
E.2 Skill Injection	18
E.3 Library Scaling	19
E.4 Out-of-Scope Behaviour	20
E.5 Robustness Analysis	20
E.6 Weight Sensitivity Analysis	21
F Prompt Templates	22
F.1 Skill-Selection Red Team	23
F.2 Task and Rubric Adversarial Review	23
F.3 LLM Judge	23
G Ethical Statement	23

A Benchmark Comparison

Table 2 places SKILL-USE against instruction-following and skill-centric benchmarks. Instruction-following benchmarks put constraints in the prompt and grade the final response, so they neither run an agent in an open environment nor test recognition. Skill-centric benchmarks such as SkillsBench and SkillLearnBench focus on skill generation, skill learning, or task success under a supplied skill, rather than autonomous retrieval and process-level use. SLBench [11] checks procedure following with the full document preloaded, so recognition is assumed and Boundary is not isolated. SKILL-USE is the only benchmark that jointly scores Trigger, Compliance, and Boundary in an open, process-level setting under progressive disclosure.

B Construction Pipeline

This section details each construction stage summarized in Section 3.3, from skill selection through the guideline loop that governs it.

B.1 Skill Selection

Four phases narrow 105,586 public GitHub skills to 79. Cost per candidate rises with each phase, so cheap filters run first. Table 3 summarises the funnel.

Benchmark	Environment		Skill-Use Dimensions				Scale
	Open Env.	Process	Skill Doc.	Trigger	Compliance	Boundary	
IFEval [12]	✗	✗	✗	✗	✓	✗	541 inst.
FollowBench [13]	✗	✗	✗	✗	✓	✗	820 inst.
ComplexBench [15]	✗	✗	✗	✗	✓	✗	1,150 inst.
InfoBench [14]	✗	✗	✗	✗	✓	✗	500 inst.
SysBench [19]	✗	✗	✗	✗	✓	✗	500 sys.
AgentIF [16]	✗	✗	✗	✗	✓	✗	707 inst.
SOPBench [22]	✗	✓	✗	✗	✓	✓	663 tasks
SkillsBench [5]	✓	✗	✓	✗	✗	✗	87 tasks
SkillLearnBench [6]	✓	✗	✓	✗	✗	✗	20 tasks
SLBench [11]	✓	✓	✓	✗	✓	✓	86 tasks
SKILL-USE (ours)	✓	✓	✓	✓	✓	✓	79 skills / 177 tasks / 1,314 items

Table 2: Positioning of SKILL-USE against instruction-following and skill benchmarks. *Open Env.*: the agent acts in a sandbox with real tools. *Process*: scoring uses the execution trajectory, not only the final answer. *Skill Doc.*: guidance ships as a progressive-disclosure skill document rather than inline prompt text. *Trigger*, *Compliance*, and *Boundary* are the three Skill-Use dimensions: recognizing and retrieving the skill, following its procedure, and respecting its prohibitions. ✓ supported, ✗ not supported. SKILL-USE is the only benchmark that covers all three dimensions in an open, process-level setting under progressive disclosure.

Phase	In	Out	Main rejection
1. Filter + cluster	105,586	15,000	low stars, duplicates
2. Fetch under quota	1,500	1,000	fetch failure
3. Rule scoring	1,000	400	generic, no rule
4. Masked-skill eval	400	80–150	no exclusive rule

Table 3: **Skill selection funnel.** Input and output counts (approximate) and dominant rejection reason for each of the four phases that reduce 105,586 candidate GitHub skills to 79. Phase 1 output is downsampled to the Phase 2 fetch quota; the final set adds retained SkillsBench and SkillLearnBench entries.

Phase 1: filter and cluster. We drop entries below 5 stars, mirror aggregators, meta-skills that teach how to write skills, names that are pure numbers or single letters or contain `test`, `example`, `template`, or `demo`, and names a prior benchmark already covers. We impose no fixed taxonomy, since a wrong one would discard dense regions of 10^5 unknown skills. Instead we split names on hyphens and underscores, cluster the top 200 tokens, and ask an LLM to propose 15 to 25 categories grounded in those clusters. Keyword rules then label the pool, with the LLM as fallback.

Phase 2: fetch under quotas. Each category gets a fetch quota by its overlap with prior benchmarks: 80 for no overlap, 50 for partial, 20 for existing coverage, capped near 1,500. Within a category we rank by name specificity, then star count, then recency, so a compound name such as `postgres-migration-patterns` outranks a short generic one. For each entry we fetch `SKILL.md` first, then `.cursorrules` or `.clinerules`, then a constraint-style `README.md`.

Phase 3: rule scoring. We score each document d on constraint density and style specificity,

$$s_{\text{den}}(d) = \sum_i w_i^{\text{den}} f_i(d), \quad s_{\text{sty}}(d) = \sum_j w_j^{\text{sty}} g_j(d),$$

and admit it only if it clears both thresholds, $s_{\text{den}}(d) \geq \tau_{\text{den}}$ and $s_{\text{sty}}(d) \geq \tau_{\text{sty}}$, with $\tau_{\text{den}} = 0.30$ and $\tau_{\text{sty}} = 0.25$. Density features $\{f_i\}$ count modal terms (MUST, NEVER, ALWAYS, REQUIRED, FORBIDDEN, CRITICAL), code examples, good-versus-bad blocks, numbered rules, length, and guardrail sections. Style features $\{g_j\}$ count named tools and commands, process constraints, counter-intuitive rules, and machine-checkable cues. Thresholding the axes separately rejects documents that are dense but generic or specific but thin. We flag hard-to-sandbox needs (GUI, GPU, paid API) for Phase 4 without rejecting.

Phase 4: masked-skill evaluation. An LLM inspects every constraint of each surviving document, and the red-team attacker prompt in Figure 15 tries to refute each proposed constraint. Let $\mathcal{C}(d)$ be its

constraint set, $\mathcal{C}_{\text{ex}}(d)$ those that survive the masked-skill check as non-generic, $\mathcal{C}_{\text{rv}}(d)$ those with a rule-based verifier (regex, file check, AST, command trace, or state diff), and $\mathcal{D}(d)$ the dimensions (tool choice, step ordering, conditional trigger, safety boundary, output format, error handling) they cover. We tier each document by

$$\text{tier}(d) = \begin{cases} \text{A} & |\mathcal{C}_{\text{ex}}| \geq 3, |\mathcal{C}_{\text{rv}}| \geq 2, |\mathcal{D}| \geq 2, \\ \text{B} & |\mathcal{C}_{\text{ex}}| \geq 2, |\mathcal{C}_{\text{rv}}| \geq 1, \\ \text{C} & |\mathcal{C}_{\text{ex}}| \geq 1, |\mathcal{C}_{\text{rv}}| \geq 1, \\ \text{D} & \text{otherwise (rejected)}. \end{cases}$$

We admit Tier A without quota; Tiers B and C enter under the Phase 2 quotas, with looser caps for domains prior benchmarks do not cover. A 10% human spot-check audits the LLM’s labels.

B.2 Task Construction

For each skill we write executable, in-scope tasks that a strong model cannot solve without the skill. The masked-skill check enforces this: we hide the document and ask whether a strong model, given only the prompt and generic practice, would produce the same behaviour. A requirement it still meets is generic and dropped; a requirement it fails is skill-exclusive and kept. We run the check twice, once per skill during selection and once per task-requirement pair during construction.

The check tests precision, not topic recognition. A good task turns on the exact choice the skill prescribes, such as tiering a rate limit per user rather than per IP at the skill’s threshold, not on whether the model knows the general topic. Any requirement a model meets from common practice carries no signal.

Prompts read like user requests and hide the procedure. We reject checklist leakage that restates the rubric axes, directional leakage that reveals the intended fix, and reverse leakage that tells an out-of-scope task not to use the skill. We also keep prescribed formats, tool names, and parameters out of the prompt, since these are themselves skill-exclusive signals.

Assets come from real projects and datasets, with reproducible preprocessing. The prompt, the manifest, and the on-disk files share one relative path, so the task resolves inside the sandbox. Reference implementations stay on the evaluator side, so a model cannot copy an answer without reading the skill.

B.3 Rubric and Judge

Each rubric has three tiers. Evaluability gates carry zero weight and only confirm a run is well formed. Skill-derived items form the Skill-Use signal and split into Compliance and Boundary. Task-outcome items record whether the requested result appeared and stay outside Skill-Use. Every scored item traces to one skill-exclusive requirement.

We prefer deterministic checks. State-diff checks read the produced files, and regex checks anchor to structured tool-call fields, not free text, so the agent’s narration cannot cause a false match. We use the scope-restricted LLM judge only when a requirement resists a reliable rule.

The judge scores one item at a time and returns yes or no, with no numeric bands. It sees the rule, one closed-form question, and only that item’s locus: the produced artifact for an output item, the trajectory for a process item. It bases its verdict on tool-call and artifact evidence rather than stated intent, and accepts any variant the skill allows. Each item is judged twice and disagreements are re-adjudicated once, using the settings in Appendix D. Table 4 lists the configuration and Figure 20 reproduces the prompt template.

B.4 Adversarial Review

Every task and rubric passes a multi-agent review before entering the benchmark. Reviewers come from different model families, so no single family’s blind spot goes unchecked. One reviewer proposes revisions, one attacks the artifact, and a separate aggregator settles disputes. Figures 16–19 reproduce the four role prompts used across the task and rubric debates.

The task review checks discriminative power. At least two reviewers from different families must confirm that each skill-exclusive point is hard to reach without the skill. If any reviewer shows a

Dimension	Operational rule
Verdict	Binary YES or NO on one requirement, with a reason of at most 25 words
Evidence locus	Output items see only the produced artifact; process items see the trajectory
Context window	Serialized tool calls, the final answer, and the first few tool results, selected per item
Evidence source	The verdict rests on tool-call and artifact evidence, not the agent’s narration
Equivalent variants	The question accepts every implementation the skill sanctions
Adjudication	Each item is judged twice and disagreements are re-adjudicated once

Table 4: **LLM-judge configuration.** Operational rules governing the scope-restricted judge: it issues a binary YES/NO verdict on one rubric item at a time, from item-specific evidence only, with each item judged twice and disagreements re-adjudicated once.

credible no-skill path to the same behaviour, the point loses its exclusive status and is down-weighted, or the task is rebuilt. Exclusivity is thus an agreement among models, not one author’s claim.

The rubric review adds executability. Each rubric passes a schema check, and each verifier is dry-run against one passing and one failing trace, so it fires exactly when it should. Reviewers then confirm that every scored item maps to a skill-exclusive behaviour; weak-agreement items are down-weighted or re-reviewed. Recurring findings feed the guideline loop in Appendix B.6.

B.5 Release Validation

We release a skill-task-rubric triple only if it passes three checks.

Executability. An asset validator confirms that the required tools, dependencies, and assets resolve inside the sandbox and that the task runs to completion. The prompt, manifest, and on-disk files must share one relative path, since a mismatch fails every model on its first tool call.

Consistency. Each task requirement must trace to a specific skill passage, and each scored item must target a signal the run can produce. We drop requirements with no supporting passage and items with no producible signal.

Rubric reliability. We replay each rubric on one passing and one failing trace. Replay flags signals that never fire, verifiers that match narration, regexes that time out under backtracking, format-sensitive failures, and leakage between the Skill-Use and task-completion scores. Regex checks stay anchored to tool-call fields under a watchdog, binary artifacts go through parsers, and the trigger gate reads deterministically. We discard any triple that fails and cannot be repaired.

B.6 Guideline Development

A written guideline governs each construction stage. The skill guideline sets the admission criteria, verification methods, and tier thresholds. The task guideline sets how a task must depend on its skill, the leakage forms a prompt must avoid, and the rules on assets and path resolution. The rubric guideline sets item classification, verifier placement across the deterministic and semantic tiers, and the safety rules for automated verifiers.

No guideline was written up front. Each began as a short draft, was applied to a batch, and every audited failure traced to either an unclear rule or an uncovered case. We then added, sharpened, or split the rule and re-applied it to a fresh batch, repeating until an audit found no new failure, then froze the guideline and regenerated downstream artifacts. The rubric guideline took the most rounds and ended with three additions: a skill-exclusivity justification on every scored item, a three-tier weight split, and a cap on total prohibition weight.

Because concrete failures drive the loop, each frozen rule traces to the counterexample behind it, as Table 5 lists.

Figures 12, 13, and 14 reproduce the three frozen guidelines as rendered English document figures.

Rule (guideline)	Failure that motivated it
No absolute paths in task prompts (task)	A prompt referred to files by an absolute path absent at runtime, failing every model on its first tool call
No severity tags or checklists in task prompts (task)	Such tags leaked the rubric structure and reduced the task to instruction-following
Reference implementations invisible at runtime (task)	A reference artifact among the inputs let models reproduce the target behaviour without reading the skill
Weights split into gate, hard, and soft tiers (rubric)	Assigning every item the maximum weight and stacking prohibitions produced identically zero scores, hiding all variance
Every scored item carries a skill-exclusivity justification (rubric)	An audit found items rewarding generic engineering; a justification field forces an auditable attribution
Output judges must not read the trajectory (rubric)	Judges credited models that described the behaviour but never wrote the output
Regexes anchor to structured tool-call fields (rubric)	Free-text regexes matched filenames from the model’s narration, giving false positives
Regexes must avoid catastrophic backtracking (rubric)	One unbounded pattern on a long trace hung the scorer, motivating a per-regex watchdog

Table 5: **Guideline rules and motivating failures.** Frozen rules in the task and rubric guidelines, each paired with the concrete construction failure that triggered its addition during the audit loop.

Skill-Use • Data Verification Console Task SF03_t7 / 177

Task Prompt user request shown to the agent

Merge three RFC PDFs into a training deck, in this exact order:
first rfc9110 pages 8-45, then rfc9000 pages 15-68, finally rfc9114 pages 5-42. Save the result next to the inputs.

Retrieved Skill • pdftk-server/SKILL.md opened by the agent at turn 2

```

MUST use pdftk with uppercase handle syntax (A=, B=, C=)
MUST validate each input via pdftk dump_data first
NEVER fall back to qpdf, pdfunite, pypdf, or ghostscript
ALWAYS name the merge output merged_<first_input_stem>.pdf

```

Agent Trajectory tool calls • 9 steps

```

t01 Bash ls && cat README.txt
t02 Skill open("pdftk-server") → procedure loaded
t03 Bash pdftk rfc9110_http_semantics.pdf dump_data | grep Pages
t04 Bash pdftk rfc9000_quic.pdf dump_data | grep Pages
t05 Bash pdftk rfc9114_http3.pdf dump_data | grep Pages
t06 Bash pdftk A=rfc9110... B=rfc9000... C=rfc9114... \
t07 Bash cat A8-45 B15-68 C5-42 output training.pdf
t08 Bash pdftk training.pdf dump_data | grep NumberOfPages

```

Rubric Checklist 4 items • scored twice

- Uses pdftk cat/output for the merge
skill-exclusive
☒ Satisfied ☐ Violated ☐ N/A
- Avoids qpdf, pdfunite, pypdf, ghostscript
boundary
☒ Satisfied ☐ Violated ☐ N/A
- Preserves the user-specified merge or...
skill-exclusive
☐ Satisfied ☒ Violated ☐ N/A
% handles reassigned alphabetically
- Uses default merged_<stem>.pdf outp...
skill-exclusive
☐ Satisfied ☒ Violated ☐ N/A
% output saved as training.pdf

2 annotators • agreement 3 / 4 **Submit** →

Figure 7: Human-verification console for rubric audit. Annotators view the task prompt, the retrieved skill procedure, and the agent trajectory on the left; on the right they mark each rubric item as satisfied, violated, or not applicable, add a note, and see the item’s skill-exclusive or boundary provenance. Five crowd annotators score one sampled task per skill, and disagreements are adjudicated by a third reviewer.

C Human Quality Audit

The automated pipeline and the adversarial review are complemented by a crowdsourced human audit of the released benchmark. We sample one task per skill, yielding 79 audited triples, and route them to five independent crowd annotators through the web console in Figure 7. For every task the console shows the user prompt, the skill procedure the agent retrieved, and the recorded trajectory, and asks the annotator to mark each rubric item as satisfied, violated, or not applicable, and to flag any item that could be met without the skill. Annotators are paid at the standard rate of 100 CNY per hour, tracked per audited task.

Across 100 sampled items scored by every annotator, pairwise agreement is 86/100 and Cohen’s $\kappa = 0.65$, indicating substantial agreement under the Landis-Koch scale. A third reviewer adjudicates the remaining items, and we repair or drop the rubric item behind each unresolved disagreement. The audit confirms that the released rubrics are reliable and that scored items track skill-exclusive behaviour rather than generic engineering, providing a human check on top of the model-based review.

D Experimental Setup

This section collects the runtime details left out of Section 4.1.

Routing. The gateway routes each model through a family-specific channel. We sample at temperature 0, top- p 1.0, and reasoning effort “high” where offered. We exclude the Gemini family from every analysis: its channel runs a schema sanitiser that renames or drops fields in the `Skill` tool, leaving the tool non-invokable across four checks.

Sandbox. Each task runs in a fresh Ubuntu-22.04 container with its own gateway. The turn limit is 120 under CC and 80 under Codex, since CC counts the `Skill()` step Codex lacks; the wall-clock limit is 1,500s. We fail and re-run a trace on (a) an upstream 5xx or 429, (b) a missing terminal result event, or (c) a non-zero exit whose first assistant message starts with “API Error”.

Trace check. We verify each trace at three layers: `llm_metrics.jsonl` for upstream status, `interaction.jsonl` for request-response pairing, and `trace.jsonl` for the CLI trajectory, and accept it only when all three agree. About 2.8% of raw traces fail and are re-run.

Judge. GPT-5.4 scores every LLM-judge item through the gateway at temperature 0, twice per item with one re-adjudication. Deterministic verifiers score in-process and are logged for replay.

E Additional Analysis

This section reports the tables and analyses behind Section 4. Figures quoted in the main text are not repeated.

E.1 Cross-Harness Behaviour

Table 6 pairs each model’s CC and Codex SU on shared tasks.

Q Findings: Cross-harness rank is stable at the top of the leaderboard but breaks in the middle, and a single-harness leaderboard therefore misreports mid-tier open-weight models.

Head models correlate strongly and preserve rank. Middle-tier open-weight models correlate weakly and several reverse sign, so a single-harness leaderboard misreports them.

Figure 8(a) shows the signed Compliance-Boundary gap $\phi_{\text{comp}} - \phi_{\text{bound}}$ is negative for every model under both harnesses, so prohibitions are easier than full procedures. Figure 8(b) reports the sham-trigger rate, the fraction of triggered traces with Compliance below 0.3; it rises most under Codex for the weakest executors, so a high trigger rate can hide shallow rule following.

E.2 Skill Injection

This subsection expands Section 4.4 with per-skill and per-model detail.

Q Findings: Preloading the skill lifts recognition rather than execution, since gains concentrate on skills with abstract names while conditional Compliance stays flat.

Figure 9 ranks skills by paired ΔSU , defined as preloaded minus native. Skills with abstract names such as `moltbook-validator`, `tdd`, and `structured-autonomy-plan` gain most, while keyword-rich names gain little or reverse. The effect is on recognition, not execution. Splitting skills by native trigger rate, hard skills drive most between-model variance, so trigger is not keyword matching. Reading conditional Compliance, sham rate, and trace length together separates Codex behaviour into

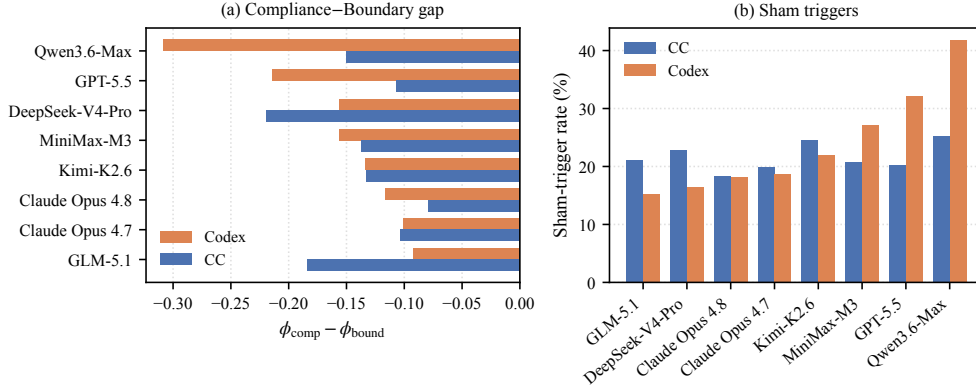


Figure 8: Execution behaviour across harnesses. (a) Signed Compliance-Boundary gap $\phi_{\text{comp}} - \phi_{\text{bound}}$ per model, negative in both harnesses, showing that prohibitions are easier to respect than full procedures. (b) Sham-trigger rate, the fraction of triggered traces with Compliance below 0.3, rising most under Codex for the weakest executors.

Model	Agreement		Mean SU		Δ
	r_{Pear}	ρ_{Spear}	CC	Codex	
Claude Opus 4.8	0.648	0.680	0.597	0.559	-0.039
Claude Opus 4.7	0.552	0.550	0.599	0.535	-0.064
GPT-5.5	0.518	0.542	0.613	0.503	-0.110
MiniMax-M3	0.424	0.425	0.501	0.446	-0.055
Qwen3.6-Max	0.302	0.337	0.318	0.238	-0.081
DeepSeek-V4-Pro	0.290	0.257	0.250	0.379	+0.129
GLM-5.1	0.285	0.246	0.267	0.421	+0.154
Kimi-K2.6	0.281	0.285	0.190	0.374	+0.185

Table 6: **Cross-harness SU agreement per model.** Pearson r and Spearman ρ between CC and Codex task-level SU, together with each harness’s mean SU and their difference $\Delta = \text{Codex} - \text{CC}$, over $n \approx 170$ paired tasks. Head models correlate strongly and keep their order, while middle-tier open-weight models correlate weakly and several reverse sign.

four profiles. Claude Opus 4.7 and 4.8 are balanced, combining high Compliance with a low sham rate. DeepSeek-V4-Pro and GLM-5.1 are selection-bound, matching head-level Compliance yet triggering natively at a low rate. GPT-5.5 and MiniMax-M3 are read-and-skip, recovering trigger but losing Compliance. Qwen3.6-Max is degenerate, producing the shortest traces, the lowest Compliance, and the highest sham rate.

E.3 Library Scaling

This subsection annotates Figures 6 (c) and (d) in Section 4.5.

Findings: As the library grows, lost SU comes almost entirely from calling no skill rather than from picking the wrong one, so scaling is bounded by recognition rather than by selection.

At $N \geq 10$ the loss is almost entirely none runs; wrong-skill calls stay below a few percent and cluster on broad descriptions like *test-driven-development* and *software-architecture*. This matches the recognition-limited pattern in Appendix E.2. Qwen3.6-Max invokes skills later than the others, so some of its none outcomes at large N are cut off by the 80-turn Codex limit; its curve is a lower bound.

Model	Claude Code (CC)				Codex			
	$\alpha=0.25$	$\alpha=0.5$	$\alpha=0.7$	$\alpha=0.75$	$\alpha=0.25$	$\alpha=0.5$	$\alpha=0.7$	$\alpha=0.75$
GPT-5.5	0.619	0.616	0.613	0.613	0.540	0.520	0.504	0.499
Claude Opus 4.8	0.593	0.595	0.597	0.598	0.565	0.561	0.559	0.558
Claude Opus 4.7	0.602	0.595	0.590	0.589	0.544	0.539	0.535	0.534
MiniMax M3	0.507	0.504	0.501	0.500	0.468	0.456	0.446	0.443
Qwen3.6 Max	0.261	0.262	0.262	0.262	0.283	0.258	0.238	0.233
GLM-5.1	0.194	0.198	0.202	0.202	0.413	0.417	0.421	0.421
DeepSeek V4 Pro	0.202	0.198	0.195	0.194	0.388	0.383	0.379	0.378
Kimi K2.6	0.182	0.186	0.190	0.190	0.375	0.375	0.374	0.374

Table 7: **Per-model SU under varied compliance weight α .** Task-level SU for each model at four settings of the compliance weight $\alpha \in \{0.25, 0.5, 0.7, 0.75\}$ under both harnesses. The default in Eq. 2 is $\alpha = 0.7$; head models keep their order across the sweep while middle-tier models shift by at most one adjacent pair.

α	CC				Codex			
	τ	ρ	shift	$\overline{\text{SU}}$	τ	ρ	shift	$\overline{\text{SU}}$
0.25	+ .86	+ .95	1	.395	+1.00	+1.00	0	.447
0.5	+1.00	+1.00	0	.394	+1.00	+1.00	0	.439
0.7	+1.00	+1.00	0	.394	+1.00	+1.00	0	.432
0.75	+1.00	+1.00	0	.394	+1.00	+1.00	0	.430

Table 8: **Ranking stability against the $\alpha = 0.7$ baseline.** Kendall τ , Spearman ρ , and the count of adjacent-pair reorderings at each swept weight, along with the harness-mean SU. Rankings hold across $\alpha \in [0.5, 0.75]$ on both harnesses; only CC at $\alpha = 0.25$ swaps a single adjacent pair.

E.4 Out-of-Scope Behaviour

Section 4.6 reports pooled avoidance scores across 53 out-of-scope tasks. This subsection breaks that pool into task types and identifies which models fail on which tasks.

Q Findings: Models agree on most out-of-scope tasks, and over-invocation concentrates on Claude Opus and GPT-5.5, while GLM-5.1 shows an all-or-nothing profile.

Models agree on most of the 53 tasks, and only 9 separate them, as Table 9 shows. All-avoid tasks carry a clean out-of-scope cue such as a mismatched framework name. All-enter tasks are document-shaped prompts whose topic overlaps the skill’s domain even though the output never needs the procedure.

Over-invocation clusters on Claude Opus and GPT-5.5, the two models closest to a skill-first harness; they share the over-invocation pattern while others differ mainly in when they refuse. GLM-5.1 either fully engages or fully ignores a skill, so its per-task distribution is bimodal, matching its Codex profile in Appendix E.2.

E.5 Robustness Analysis

To test whether a single scoring pass is stable enough for model-level comparison, we freeze the CC traces and rescore a stratified subset of 83 aligned (*task, model*) observations three times with the same judge, rubric, and prompt, so all variation comes from the judge rather than the agent.

Q Findings: The leaderboard survives repeated scoring, and judge noise is confined to mid-range traces rather than to clear wins or clear failures.

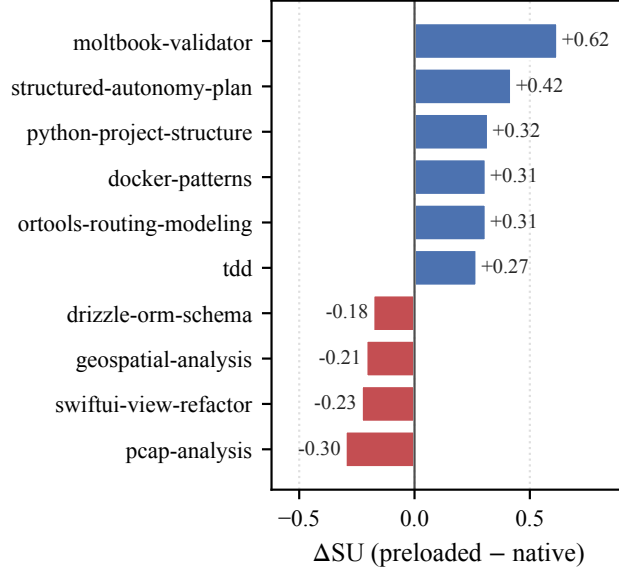


Figure 9: Per-skill gain from preloading. Paired ΔSU , defined as preloaded minus native, per skill. Abstract names gain most because native selection misfires, while keyword-rich names gain little or reverse. The effect operates on recognition, not execution.

Task	Avoid	Over-invokers
docs_only_negative	5/6	GPT-5.5
api_security_inventory	5/6	Opus 4.7
frontend_backend	4/6	Opus 4.7, GPT-5.5
go_release_note	4/6	Opus 4.7, GPT-5.5
airflow_release_notes	5/6	Opus 4.7
reading_inventory	5/6	GPT-5.5, GLM
python_email	4/6	Opus 4.7, GPT-5.5
warehouse_inventory_note	3/6	Opus 4.7, GPT-5.5, GLM
crypto_market_landscape	4/6	Opus 4.7, GPT-5.5

Table 9: **Out-of-scope disagreement tasks.** The 9 of 53 tasks where models split. Avoid is the number of the six evaluated models that correctly refrain from invoking the skill, and Over-invokers names those that trigger anyway. Claude Opus and GPT-5.5 account for most inappropriate invocations.

Figure 10 shows the three rescores preserve the leaderboard; the few swaps involve models already within a narrow SU gap. Averaged over about one hundred tasks per model, the model-mean uncertainty falls far below the tier gaps in Table 1.

Table 10 reports per-metric agreement. Compliance is the most stable continuous component; Boundary and task completion vary more, and Boundary has fewer observations because early records omit the field. Figure 11 shows judge disagreement peaks for mid-range traces and is lowest at clear successes and failures. Section 4.2 makes no ordering claim between adjacent mid-range models, so this noise does not threaten the conclusions. Section 3.1 shows that the trigger gate is deterministic, so all variation comes from judge stochasticity.

E.6 Weight Sensitivity Analysis

The default SU weight in Eq. 2 is $\alpha = 0.7$, reflecting that reading and following a skill is the primary target while boundary adherence is a secondary check. We test whether this choice is decisive by sweeping $\alpha \in \{0.25, 0.5, 0.7, 0.75\}$ on both harnesses, omitting $\alpha \in \{0, 1\}$ because they collapse

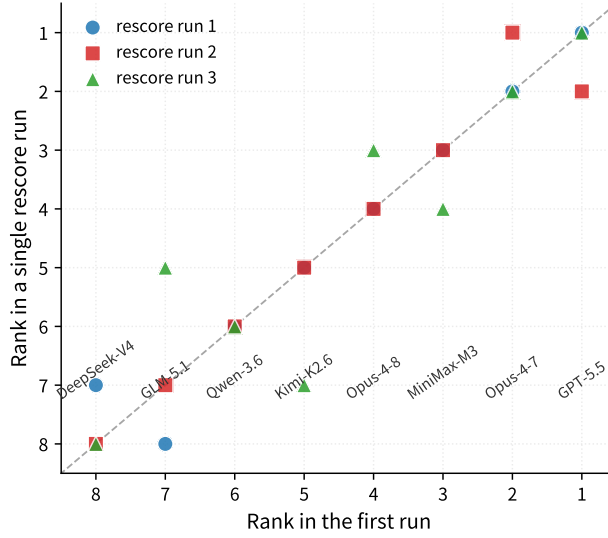


Figure 10: Ranking stability across three independent rescoring runs. Each dot plots a model’s rank in one rescoring against its rank in another, and near-diagonal alignment indicates a preserved leaderboard. The two off-diagonal pairs involve models within 0.01 SU of each other.

Metric	n	ICC(2,1)	Within std
Skill-Use	83	0.530	0.141
Compliance	83	0.604	0.116
Boundary	39	0.471	0.186
Task completion	83	0.478	0.176

Table 10: **Repeated-score agreement per metric.** ICC(2,1) and within-observation std across three rescoring runs of the aligned CC subset. Compliance is the most stable continuous component, and Boundary has fewer observations because early baseline records omit this field.

SU to a single component. Table 7 lists per-model SU at each α and Table 8 the ranking stability against the $\alpha = 0.7$ baseline, on the same samples as Section 4.2.

Findings: Model rankings are stable across a wide range of compliance weights, so the default $\alpha = 0.7$ is not a fragile setting.

Both harnesses give identical orderings for $\alpha \in [0.5, 0.75]$. Only CC at $\alpha = 0.25$ reorders, confined to two adjacent pairs that cross no tier boundary. Absolute SU shifts under reweighting are one to two orders of magnitude smaller than the tier gaps, so the qualitative claims of Section 4.2 hold across the range.

F Prompt Templates

This section reproduces the prompt templates behind the three review roles that guard benchmark quality: the red-team attacker used during skill selection, the four proposer-attacker-aggregator prompts used in the multi-vendor task and rubric debates, and the scoring-time LLM judge. Each prompt is shown in its English form, with concrete file paths and Python placeholders stripped for readability. Templates for skill categorisation, masked-skill checking, task generation, and rubric generation are omitted; those stages are described in Appendices B.1–B.3, and their template text is not required to reproduce the results.

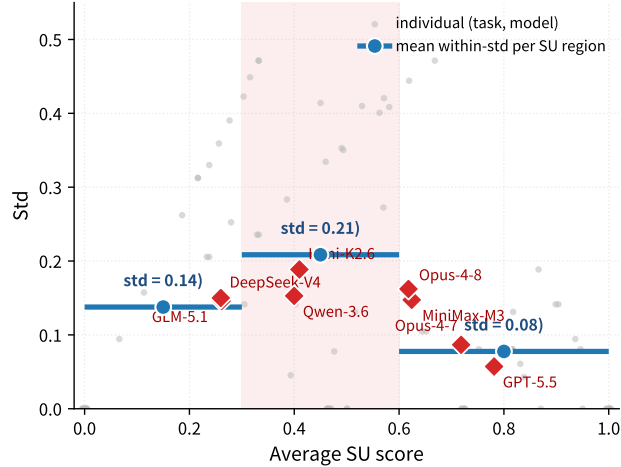


Figure 11: Judge noise across the SU range. Within-observation std over three rescoring against SU. Noise peaks in the middle band and is lowest at both extremes, where the paper’s strong-comparison claims sit.

F.1 Skill-Selection Red Team

Appendix B.1 describes the skill-selection stage, where every candidate constraint is challenged by a red-team attacker that tries to refute its skill-specificity, verifiability, triggerability, non-redundancy, and gradability. Constraints that survive the attack are kept; the rest are revised or dropped. Figure 15 reproduces the attacker prompt together with its scoring rubric and the accept/revise/reject verdict.

F.2 Task and Rubric Adversarial Review

Every task and every rubric goes through the multi-vendor debate described in Appendix B.4. Each debate uses two role prompts, a reviewer that proposes revisions from a fixed list of review axes, and an aggregator that converts the debate transcript into a per-field or per-rule edit list. Figure 16 reproduces the task reviewer prompt, which covers six axes: skill-exclusive discrimination, description safety, real data and scenario, trigger design, evaluability, and asset consistency. Figure 17 reproduces the task aggregator prompt, which emits an actionable field-level edit list under hard constraints on which YAML fields may be edited, five red-line failure modes, and a quality gate. Figure 18 reproduces the rubric reviewer prompt, which covers four axes: skill-exclusive judgement, evaluation reliability, task-completion quality, and signal correctness. Figure 19 reproduces the rubric aggregator prompt, which issues a per-rule action table (keep, remove, revise, merge, downgrade) together with a must-fix list and a quality gate that reports signal purity, verifier reliability, and LLM-judge share.

F.3 LLM Judge

The scope-restricted judge scores one rubric item at a time and returns a binary verdict under the configuration in Table 4. Figure 20 reproduces the full template: the rule identifier and specification, one closed-form YES/NO question, and a trace context assembled from serialised tool-call steps, the first few tool-call results, and the final answer.

G Ethical Statement

SKILL-USE is built entirely from public GitHub skills and open-source datasets under licences that permit redistribution, and contains no personally identifying information. Human audits use crowd annotators who are informed of the research purpose and compensated above the local minimum wage. All runs execute in isolated containers, and we release the rubrics and prompts so that external reviewers can reproduce and audit our results. The benchmark measures skill use in an agentic harness and is not a certification of safety or deployment readiness.

Skill Selection Guideline Goal Select 80-150 seed skills from 105,586 public GitHub skills. The selected set should be discriminative, verifiable, executable in a Docker sandbox, and diverse across at least 15 domains. Good Skill Profile A qualified skill must satisfy three conditions at the same time. - Discriminative: it contains skill-specific constraints that a strong model is unlikely to follow without reading the skill. - Verifiable: each important constraint can be checked by regex, AST, file checks, command traces, or state diffs. - Executable: the required workflow can run in an isolated sandbox without GUI, GPU, paid APIs, or external cloud services. Preferred constraints describe process behavior: tool choice, step order, argument use, intermediate artifacts, error handling, or safety boundaries. Avoid generic advice such as meaningful names, clean comments, best practices, or final-output-only requirements. Coverage Policy The target is at least 15 domains. Categories are not fixed in advance; they are discovered from the GitHub metadata. The priority order is: first, domains not covered by the existing benchmark; second, new sub-directions inside already covered domains; third, deeper coverage only when the candidate quality is very high. Existing coverage includes document processing, databases, web frontend, testing, security, DevOps, networking, optimization, signal processing, systems, programming languages, and Git workflows. Pipeline Phase 1. Metadata screening and data-driven classification. Remove obvious noise, analyze name token frequencies, cluster co-occurring terms, ask an LLM to form 15-25 natural categories, and map about 15,000 candidates into those categories. Phase 2. Quota-based retrieval. Fetch SKILL.md, cursorsrules, clinerules, or rule-heavy README files from top candidates in each category. Allocate larger quotas to categories with little or no overlap with the current benchmark. Target 1,000-1,500 full-text skills. Phase 3. Automatic constraint-quality scoring. Score both constraint density and constraint style. Density uses strong modal words, code examples, good/bad contrasts, structured rules, length, and guardrails. Style uses tool specificity, process constraints, anti-prior behavior, and executable verification clues. Phase 4. LLM deep evaluation and tiered selection. For each full skill, list skill-specific constraints, run a masked-test prediction, assign verifier types, label constraint dimensions, choose a tier, and record primary and secondary categories. Hard Filters and Ranking Remove entries with very low stars, mirrored aggregation repositories, meta-skills about writing skills, obvious tests/templates/examples, and exact duplicates of already covered benchmark skills. Within each category, rank by name specificity first, then stars, then recent maintenance. Specific names such as postgres-migration-patterns are preferred over broad names such as database. Tiering and Final Checks Tier A requires at least three skill-specific constraints, at least two rule-verifiable constraints, and at least two heterogeneous constraint dimensions. Tier B requires at least two skill-specific constraints and one rule-verifiable constraint. Tier C is only used for missing categories. The final audit checks domain coverage, overlap with the existing benchmark. Tier A ratio, at least two verified skill-specific constraints per skill, low redundancy between selected skills, and less than 10% sandbox-infeasible skills. Artifacts phase1_classified.json: classified metadata candidates. phase2_with_content.json: retrieved full-text skills. phase3_scored.json: density and style scores. final_selected.yaml: selected skills with constraints, verifier labels, categories, and feasibility notes. category_report.md: per-category distribution and quality report.	
--	--

Figure 12: **Skill-selection guideline.** Frozen rules governing which GitHub skill documents are admitted into the benchmark, including admission criteria, verification methods, and tier thresholds.

Task Construction Guideline
Guiding Principle Use real data to create real scenarios that test real skill-following. A task is valid only when performance depends on whether the model reads and applies the skill.
Two Core Properties - Skill extension: solving the task well is strongly related to reading the skill. If the task is easy without the skill, it should be redesigned. - Real scenario plus real data: assets come from real webpages, public APIs, open-source datasets, or real project artifacts. Do not invent toy files from scratch.
Masked Test and Skill-Exclusive Point Ask: if the skill document is removed, can a strong model still solve the task easily and in the same way? If yes, the task does not measure skill-following. Each task should explicitly mark at least one skill-exclusive point: what the skill requires, what a no-skill model would naturally do, and where the task forces the divergence. Discrimination should come from precision and normative stance, not topic discovery. Strong models already know common migrations, common security bugs, common code-review formats, and any behavior directly stated in the description.
Reference Isolation Reference implementations and gold answers are evaluator-side assets only. They must not appear in model-visible input directories. If a visible input contains a reference file, the model can imitate it and bypass the skill. Allowed locations are rubrics/ or assets/<TASK_ID>/expected/. Model-visible assets/<TASK_ID>/input/ should contain only the task inputs.
Real Data and Asset State Real assets may be preprocessed, but the content itself should come from real sources. Rotating, encrypting, splitting, merging, or sampling real files is allowed. Generating fake document bodies, fake forms, or fake datasets is not allowed. The asset state must match the description exactly. If the description says a PDF is rotated, encrypted, fillable, recently changed, or missing metadata, the file must actually have that state.
Path Rules Description paths, initial_state.assets[], path, and disk paths must be identical from the model's perspective. Use relative runtime paths such as assets/<TASK_ID>/input/file.pdf. Do not use /root, /workspace, or other absolute paths in descriptions. Output paths should also be relative, such as output/output.pdf.
Description Design Write the task as a natural user request, not as a technical checklist. Do not reveal tools, commands, parameters, algorithms, exact thresholds, skill-defined formats, or implementation strategy. Avoid three leakage modes: checklist leakage that lists evaluation axes, directional leakage that gives the answer skeleton, and reverse leakage that tells the model not to trigger a skill. Format injection is forbidden. Skill-defined headings, severity labels, templates, or output schemas should remain skill-exclusive signals and should be checked by the rubric, not stated in the description. File-level hints can also leak the solution by narrowing the search space. Mention a specific source file only when a real user would naturally provide it.
Trigger Design Positive tasks are in-scope situations that should use the skill and contain at least one skill-exclusive point. Control tasks may change conditions to trigger a different branch. Negative tasks are superficially related but out of scope, and they should be confusing rather than obviously irrelevant. The single source of truth for task_kind is expected_trigger in the task YAML. Scores, merge pipelines, and CSV files should derive task_kind from the YAML rather than hard-code it.
Evaluability Design the task together with its future scoring evidence. Separate baseline correctness signals from skill-following signals. Baseline correctness checks whether the task artifact is basically right; skill-following signals check whether the skill-specific behavior was followed. Intermediate artifacts such as runnable scripts, command logs, or reports are useful only when they are observable and when the skill itself supports treating them as skill-following evidence.
Task Template and Checks A task YAML should include task_id, skill_id, natural-language description, initial_state assets with source, real_state and preparation, expected_trigger, skill_constraints_tested, and notes separating baseline correctness from skill-following signals. Notes should describe common model errors at concrete code-line or command-step granularity, not as vague labels. Each task constraint must have clear support in the skill document.
Scale, Gradient, and Anti-Patterns Prefer multiple tasks per skill when the skill has independent scenarios, branches, or difficulty levels. Do not stop at the minimum count if useful diversity remains, and do not add homogeneous tasks just to increase count. Avoid toy generated assets, no-skill-easy tasks, descriptions that name tools or parameters, mismatched asset state, missing negative cases, unreproducible preprocessing, absolute paths, reference answers in model-visible inputs, public-knowledge discrimination, unsupported constraints, and vague notes.

Figure 13: **Task-construction guideline.** Frozen rules governing how each task must depend on its skill, forbidden leakage patterns in prompts, and requirements on assets and path resolution inside the sandbox.

Rubric Design Guideline
Core Question Can this rule accurately evaluate whether the model followed the skill? The rubric has two separate goals: score skill-following precisely, and record task completion as a side channel with weight 0.
Three Tests for Every Rule - Skill-exclusive test: hide the skill and read only the task description. If a strong model would still do the behavior from common knowledge, the rule is not a skill-following signal. - Verifiability test: given actual artifacts or tool calls, the rule should reliably decide followed vs not followed. - Accuracy test: the rule should avoid false positives and false negatives caused by vague signals, self-report text, imprecise paths, or overly narrow accepted variants.
Verifier Placement Prefer state-diff for final artifact contents. Use regex for fixed tool-call names or arguments, anchored to structured tool-call fields rather than agent self-report. Use llm-judge only for semantic, conditional, or multi-step judgments that deterministic checks cannot capture. Verification should focus on results and real evidence. A command string in the trace is weaker than the artifact it actually produces, and a model statement about intent is not evidence of execution.
Spec-Signal Alignment Every atomic requirement in spec must map to a detectable condition in signal. AND must not become OR. A per-item requirement must not become a global mention check. A surface token match must prove the intended semantic chain. The check procedure is: split the spec into atoms, find a signal condition for each atom, preserve conjunctions and disjunctions, and match quantifiers such as every, all, and at least N.
False Positives and False Negatives Common false positives include regex matching prompt text, skill text, comments, errors, output paths, or serialized plans instead of real tool calls. Common false negatives include missing accepted variants, quote or line-continuation differences, and large conjunctive rules that should be split. Regex rules should be scoped to command or tool-call context and should cover common equivalent forms. LLM judges should explicitly ignore self-report and judge only the specified evidence.
Signal Separation Gate: provenance=gate, weight=0. It checks whether the trace contains an observable attempt and is eligible for scoring. Skill-following: provenance=skill, weight>0. It is the main scored signal and must pass the skill-exclusive test. Task completion: provenance=task, weight=0. It records whether the base task was done and must not enter the Skill-Use score.
Rule Types and Weights Skill-following rules may check required tools, required arguments, lessons, step order, termination discipline, forbidden tools, forbidden arguments, or boundaries. Headline skill-exclusive rules get weight 1.0. Important workflow or parameter requirements get 0.5-0.75. Soft details get 0.25. Gate and task-completion items always use weight 0. Use a three-layer structure: gate for evaluability, a small number of must-do rules for core behavior, and soft bonus rules for details. Must-not weight should not dominate must-do weight, and no single rule should monopolize the score.
Skill-Exclusive Justification Every provenance=skill item should include skill_exclusive_reason. The field explains what a no-skill model would likely do, what the skill requires instead, and why the conflict is specific to the skill. If the reason cannot be written concretely, the rule is probably generic practice or task completion. Delete it or set weight 0 outside the compliance or boundary denominator.
LLM-Judge Protocol A judge query should be closed-form YES/NO, ask one thing, anchor the evidence source, provide acceptable and unacceptable examples, and limit the context window. For final-output rules, judge_context_window should contain only final_answer. If process behavior must be checked, use trace locus with deterministic evidence when possible. Do not let final-output judges read serialized steps, because plans and failed attempts create false positives.
State-Diff and Binary Artifacts State-diff paths must point to exact files, not directories. file_must_contain and file_must_not_contain can check text outputs. Binary files such as PDF, DOCX, XLSX, and PPTX should be parsed by dedicated tools rather than raw text regex.
Regex Safety Avoid catastrophic backtracking on large traces. Do not use greedy [s/S]* or .* followed by literals over 100KB text. Prefer non-greedy quantifiers, fixed anchors, bounded ranges such as :{0,2000}, or step-level matching. Every regex should be tested on large dummy text and should finish quickly. Timeout protection is only a fallback; safe patterns should be written at rubric design time.
Calibration and External Dependencies After real scoring, lower the weight of rules that all models fail, because they do not differentiate models. Prefer workflow or behavior-pattern rules as headline items when they separate models better than simple tool-choice rules. Do not reward the same behavior twice through overlapping rules. If a skill cites helper scripts, the scripts must exist and be executable. For live URLs, score the fetch behavior or use a frozen snapshot rather than depending on dynamic text.

Figure 14: **Rubric-design guideline.** Frozen rules governing rubric item classification, verifier placement across the deterministic and semantic tiers, and safety constraints for automated verifiers.

Role and Perspective

You are the red-team attacker for the skill filter. You come from a different vendor than the Rewriter and reason as an engineer who has never read this skill. Your job is to decide whether the proposed constraints are truly skill-specific or would be produced by any strong model given the task description alone.

Inputs

- The original skill document.
- The Rewriter's revised skill with a table of 6-10 constraints and their claimed skill-specific status.

Per-Constraint Attack

For every constraint in the revised skill, answer five questions in order.

- Skill-specific: without reading this skill, using only general knowledge and common domain sense, what would a strong model naturally do on a task in this area? Is that different from what the skill requires? Rate the difference as almost never the same, sometimes different, or basically identical.
- Verifiability: which concrete verifier would catch a violation, using regex on tool calls, state-diff on the artifact, or a bounded LLM judge? If none can be described, verifiability is weak.
- Triggerability: how contrived must a task be to force this constraint to matter? If the constraint only fires in artificial setups, it is not useful.
- Redundancy: does it overlap with another constraint on the list, so that following one already implies the other?
- Granularity: can the constraint be scored on a graded scale, or only as a binary pass or fail?

Global Scoring

Score six dimensions on a 1-5 scale and report the total: verifiability, task-synthesis feasibility, skill-specificity, rule heterogeneity, granularity, and sandbox executability.

Verdict

- Total at least 4: accept, the skill may enter task synthesis.
- Total between 3 and 4: revise, list every constraint that must change and the exact direction of change.
- Total below 3: reject, a major rewrite is required before the skill is considered again.

Output Format

For each constraint report a short verdict block with the five per-constraint answers, the total score, and the accept/revise/reject decision. When revision is requested, give a concrete edit for every flagged constraint.

Figure 15: Skill attacker prompt. Per-constraint attack protocol used during the skill-selection filter to decide whether each proposed constraint is truly skill-specific, verifiable, triggerable, non-redundant, and gradable, together with the global scoring rubric and the accept/revise/reject verdict.

Role

You are a Skill-Use benchmark task-quality reviewer taking part in a multi-vendor debate. You share the round with two reviewers from different providers. The benchmark measures whether the agent behaves according to the skill, not whether it happens to solve the task, so a good task must fail without the skill, use real data, and hide the answer in the description.

Inputs Available

- The task YAML, including description, initial state, expected trigger, tested constraints, and notes.
- The full skill document.
- The frozen task-construction guideline.
- The listing of assets that actually exist on disk for this task.

Six Review Axes

- Skill-exclusive discrimination (most important). Remove the skill, keep only the description, and ask what a strong model would do by default. If the natural default already matches the skill requirement, discrimination is zero. For every claimed skill-exclusive point, state explicitly what a no-skill model would do and what the skill demands, and separate coincidental agreement from necessary divergence.
- Description safety. Check that the description never leaks tool names, commands, arguments, numeric thresholds, skill-defined formats, or implementation strategy. The description should read like a real user request, not an instruction-following script.
- Real data and real scenario. Assets must come from real sources rather than fabricated toy files, and their on-disk state must match the description (rotated, encrypted, missing metadata, and so on).
- Trigger design. Positive tasks must sit on a skill-exclusive divergence. Negative or control tasks, when present, must look superficially related to the skill and still be out of scope.
- Evaluability. Every claimed constraint must have an observable verifier signal, and skill-following signals must be separable from base task completion signals. Even a truncated trace should still expose skill-following behaviour up to the truncation point.
- Asset consistency. Input paths in the description and in initial_state must match the disk listing exactly; discrepancies are high severity.

Hard Non-Editables

The reviewer must not propose changes to skill.md, task_id, tools_available, expected_trigger, level, or skill_id. Toy assets fabricated from scratch remain forbidden. Task completion signals must never be relabelled as skill-following signals.

Output Format

For each of the six axes give a pass, warning, or failure verdict with concrete evidence quoted from the task YAML. For every warning or failure, provide a field-level fix in the form change field X from A to B. End with a global verdict on whether the task may enter rubric synthesis and, if not, list the blocking issues.

Figure 16: Task reviewer prompt at initial debate round.

Role

You are the task-quality Aggregator. You read the multi-round debate among reviewers and produce an actionable ruling that the executor agent can apply verbatim.

Hard Constraints

- You may only edit the task YAML fields description, skill_constraints_tested, notes, and initial_state.
- Never modify task_id, level, skill_id, tools_available, expected_trigger, or the skill document itself.
- Path issues must be fixed by rewriting description or initial_state to match real disk locations. Only input paths are in scope; output paths remain free.
- Answer leakage in the YAML may be trimmed or paraphrased inside the YAML only. Runtime logic is out of scope.
- Every proposed change must be concrete at the field level and directly executable.
- Fabricating toy assets is forbidden.

Red Lines

Any of the following forces the item into the must-fix section.

- No skill-exclusive point exists. The task cannot enter the benchmark.
- The description leaks a tool, command, or answer. Rewrite the description.
- Paths disagree between description, YAML, and disk. Fix the paths.
- Asset state contradicts the description. Fix the asset state.
- A claimed exclusive property fails your own masked test. Correct or remove it.

Ruling Principles

- Positions that survive across debate rounds are treated as consensus and enter must-fix.
- Disagreements are recorded with each side's argument and your own ruling with justification.
- Later-round revisions of a reviewer's position override the initial-round view.
- For every skill-exclusive dispute, perform the masked test yourself using the skill text: what would you do without the skill, and does it truly diverge from the skill requirement?

Output Format

Return four labelled sections.

- Must-fix. For every entry list the affected field, the diagnosed problem, the exact rewrite, the models and rounds that agreed, and a consensus tag of high or medium.
- Suggested. Same schema, only for reviewer suggestions that did not reach consensus but you still recommend.
- Disagreements. Record the dispute, each side's argument, your ruling with reasoning, and a consensus tag of low.
- No-change. Reviewer proposals you reject, each with a short rejection reason.

Finish with a quality gate that judges skill-exclusive discrimination, description safety, real-data status, asset consistency, and whether the task may enter rubric synthesis.

Figure 17: Task aggregator prompt.

Role

You are a Skill-Use benchmark rubric-quality reviewer in a multi-vendor debate. The core question is whether each rule truly measures whether the agent followed the skill, not whether the task happens to be completed.

Inputs Available

- The rubric YAML with every rule id, spec, provenance, verifier, and signal.
- The matching task YAML.
- The full skill document.
- The frozen rubric-design guideline.

Four Review Dimensions

- Skill-exclusive judgement. Hide the skill and read only the description. If a strong model would still perform the behaviour by default, the rule is not a skill-following signal and should be removed. For every skill rule, write in one line what a no-skill model would do and what the skill requires, and confirm that the two disagree.
- Evaluation reliability. Prefer verifying the result on the final artifact over verifying process text in the trace. State-diff paths must be precise. Regex must anchor to real tool-call contexts, not model self-report or comments, and must cover equivalent forms allowed by the skill. LLM-judge queries must be closed-form YES or NO, ask one thing, require evidence from tool-call outputs, provide acceptable and unacceptable examples, and use a non-empty signal.
- Task-completion quality. Rules with provenance task must verify that the artifact is genuinely correct, not merely present. Text artifacts use state-diff, binaries use dedicated parsers, and every task-completion rule must have weight zero.
- Signal correctness. The signal field is the actual executable code for the verifier. For every rule ask whether the signal will pass a model that truly follows the skill, fail a model that follows the default behaviour, and reject a model that only mentions the required keywords. Check that state-diff paths point to model outputs rather than to skill sources, that regex escapes are correct, and that an LLM-judge rule reducible to state-diff or regex is downgraded to the deterministic verifier.

Structural Checks

- No two rules should score the same behaviour. Merge duplicates.
- Task-completion behaviour must not be relabelled as skill provenance.
- Headline skill-exclusive rules must carry the highest weight; the LLM-judge share of scored weight should remain below thirty percent.

Output Format

For every rule id output a short block containing skill-exclusive yes or no with reasoning, evaluation reliability with concrete false-positive or false-negative sources, signal correctness with a specific rewrite when needed, and a final action of keep, remove, revise, merge, or downgrade. End with an overall verdict on signal purity and reliability and on whether the rubric may enter the executor stage.

Figure 18: Rubric reviewer prompt at initial debate round.

Role

You are the rubric-quality Aggregator. You read the multi-round rubric debate and issue a rule-by-rule ruling that the executor agent can apply to the YAML.

Guiding Principle

The rubric grades whether the agent followed the skill, not whether the task was completed. Every rule-level ruling must answer the same question: is this rule measuring skill-following, or is it measuring something else?

Per-Rule Judgement Criteria

Apply the criteria in order.

- Provenance correctness. Was skill, gate, or task chosen correctly? Task-completion signals must not sit in the skill bucket.
- Masked test. Hide the skill and inspect only the task description. If the behaviour can still be inferred, the rule is not skill-following and is downgraded or removed.
- Discrimination. If a no-skill model would naturally satisfy the rule, its weight is lowered or the rule is removed.
- Verifier reliability. Regex must check results rather than trace text and must not match self-report, comments, or failed attempts. LLM-judge must be closed-form and must require evidence from tool-call outputs. Signals must be non-empty, and binaries must use dedicated parsers.
- Signal correctness. The signal field must actually detect the behaviour named in spec. State-diff paths must point to model outputs, regex must cover equivalent forms without escape errors, and LLM-judge queries must ship acceptable and unacceptable code-level examples. When state-diff or regex can express the same check, the rule is downgraded from LLM-judge.
- Redundancy. Rules that measure the same behaviour are merged.
- Signal separation. Task-completion behaviour must never enter the scored skill-following bucket.

Ruling Principles

- Positions that persist across rounds enter must-fix as high-consensus items.
- Disagreements record each side's argument together with your own ruling.
- Later-round revisions override earlier positions.
- For skill-exclusive disputes, perform your own masked test against the task description.
- Rules that every reviewer says all models will satisfy have zero discrimination and cannot be skill-following signals.

Output Format

Produce four sections plus a quality gate.

- Per-rule ruling table. One row per rule with columns rule id, action (keep, remove, revise, merge, downgrade), consensus (high, medium, low), one-sentence reason, and the concrete edit (field, before, after).
- Must-fix. Consensus items and structural failures, each with an executable edit.
- Suggested. Non-consensus improvements you still endorse.
- Disagreements. Persistent low-consensus disputes with your ruling.

The quality gate reports skill-exclusive signal purity, verifier reliability, signal correctness, the presence of at least one skill-exclusive rule (otherwise the skill must be strengthened first), the LLM-judge share of scored weight with a warning above thirty percent, the count of downgradable LLM-judge rules, and whether the rubric may enter the executor stage.

Figure 19: Rubric aggregator prompt.

Role

You are a strict judge for the Skill-Use benchmark. Given a single skill-following rule and an agent execution trace, decide whether the rule was followed. The verdict is used only when the rubric's deterministic verifiers cannot decide.

Prompt Template

The judge receives the rule id, the rule specification, one closed-form yes or no question, and a trace context assembled from three configurable slots.

- Serialized steps. One tool call per line in the format tool:args:intent, indexed by step.
- First tool_results. The results of the first N tool calls, truncated to five hundred characters each, when the signal requests them.
- Final answer. The final assistant response, truncated to two thousand characters, when the signal requests it.

Prompt Body

The prompt sent to the judge model has the following stable structure.

You are a strict judge for a Skill-Following benchmark.

You will read an agent execution trace and answer ONE yes/no question about whether a specific skill rule was followed.

Rule (id=<rule_id>):

<rule specification>

Question to answer (output ONLY "YES" or "NO" on the first line, then on the second line a short reason no longer than 25 words):

<closed-form question>

Trace context:

<serialized steps and, if requested, first tool_results and final answer>

Answer Contract

- Line one must contain only YES or NO. Any other token invalidates the verdict and the item is recorded as verifier failure.
- Line two is a reason of at most twenty-five words that quotes concrete trace evidence rather than restating the rule.
- The judge must ignore agent self-report about intent and rely only on observable tool calls, tool results, and the final answer.
- The verdict passes when it matches the signal's expected polarity, which the rubric author specifies as YES or NO through judge_expected.

Operational Notes

- The default judge model is a strong external model held constant across all evaluated agents to avoid same-family bias.
- The judge is called through a bounded API call with retries, a five-minute read timeout, and thread-local sessions.
- Verifier metadata records the raw verdict, the parsed YES or NO, the expected polarity, and the truncated reason, so every LLM-judge decision remains auditable.

Figure 20: The LLM-judge prompt.